

Firewall Exploration

April, 2026

Contents

1	Overview	2
2	Task 1: Implementing a Simple Firewall	2
2.1	Task 1.A: Simple Kernel Module	2
2.2	Task 1.B: Netfilter Firewall	2
2.2.1	Sub-task 1: Block UDP to 8.8.8.8:53	2
2.2.2	Sub-task 2: Hook printInfo to All Five Hooks	3
2.2.3	Sub-task 3: Block Ping and Telnet to VM	3
3	Task 2: Stateless Firewall with iptables	4
3.1	Task 2.A: Protecting the Router	4
3.2	Task 2.B: Protecting the Internal Network	4
3.3	Task 2.C: Protecting Internal Servers	5
4	Task 3: Stateful Firewall	5
4.1	Task 3.A: Connection Tracking	5
4.2	Task 3.B: Stateful Firewall Rules	7
5	Task 4: Limiting Network Traffic	7
6	Task 5: Load Balancing	8
6.1	nth Mode (Round-Robin)	8
6.2	Random Mode	9
7	Conclusion	9

1 Overview

This lab explores firewall implementation and configuration through two approaches: building a custom packet-filtering firewall using Linux Loadable Kernel Modules (LKM) and Netfilter hooks, and configuring the built-in `iptables` firewall. Topics covered include stateless and stateful packet filtering, connection tracking, traffic limiting, and load balancing.

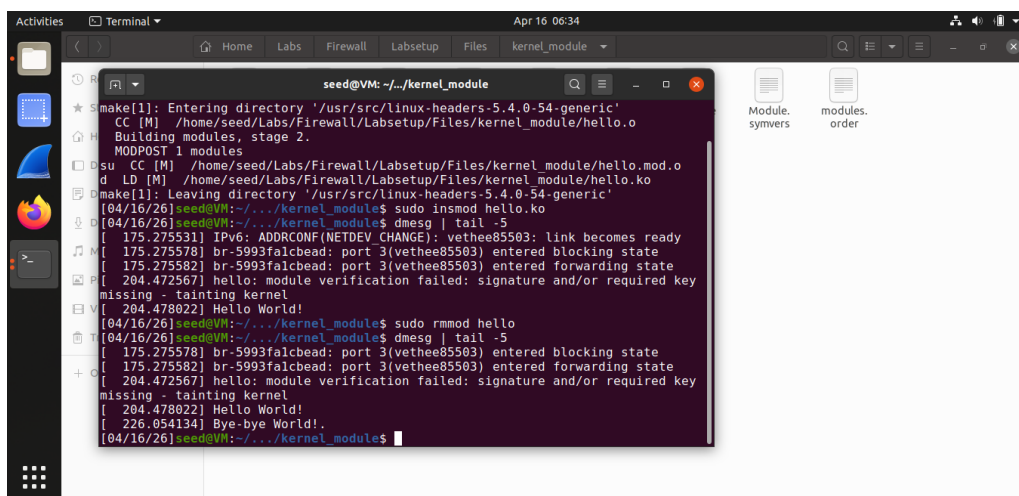
Host	IP Address
Attacker	10.9.0.1
Host A	10.9.0.5
Router	10.9.0.11 / 192.168.60.11
Internal Host 1	192.168.60.5
Internal Host 2	192.168.60.6
Internal Host 3	192.168.60.7

Table 1: Lab network topology.

2 Task 1: Implementing a Simple Firewall

2.1 Task 1.A: Simple Kernel Module

We compile and load the provided `hello.c` LKM to verify the kernel module workflow:



```
seed@VM: ~/kernel_module
$ make
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-54-generic'
CC [M] /home/seed/Labs/Firewall/Labsetup/Files/kernel_module/hello.o
Building modules, stage 2.
MODPOST 1 modules
$ sudo insmod hello.ko
[04/16/26]seed@VM:~/kernel_module$ dmesg | tail -5
[ 175.275531] IPv6: ADDRCONF(NETDEV_CHANGE): vethee85503: link becomes ready
[ 175.275578] br-5993falcbad: port 3(vethee85503) entered blocking state
[ 175.275582] br-5993falcbad: port 3(vethee85503) entered forwarding state
[ 204.472567] hello: module verification failed: signature and/or required key
missing - tainting kernel
[ 204.478022] Hello World!
[04/16/26]seed@VM:~/kernel_module$ sudo rmmod hello
[04/16/26]seed@VM:~/kernel_module$ dmesg | tail -5
[ 175.275578] br-5993falcbad: port 3(vethee85503) entered blocking state
[ 175.275582] br-5993falcbad: port 3(vethee85503) entered forwarding state
[ 204.472567] hello: module verification failed: signature and/or required key
missing - tainting kernel
[ 204.478022] Hello World!
[ 226.054134] Bye-bye World!
[04/16/26]seed@VM:~/kernel_module$
```

Figure 1: `dmesg` output showing `Hello World!` on load and `Bye-bye World!` on removal, confirming the LKM lifecycle works correctly.

2.2 Task 1.B: Netfilter Firewall

2.2.1 Sub-task 1: Block UDP to 8.8.8.8:53

We load `seedFilter.ko`, which hooks `blockUDP` to the `NF_INET_POST_ROUTING` hook and drops DNS queries to 8.8.8.8:

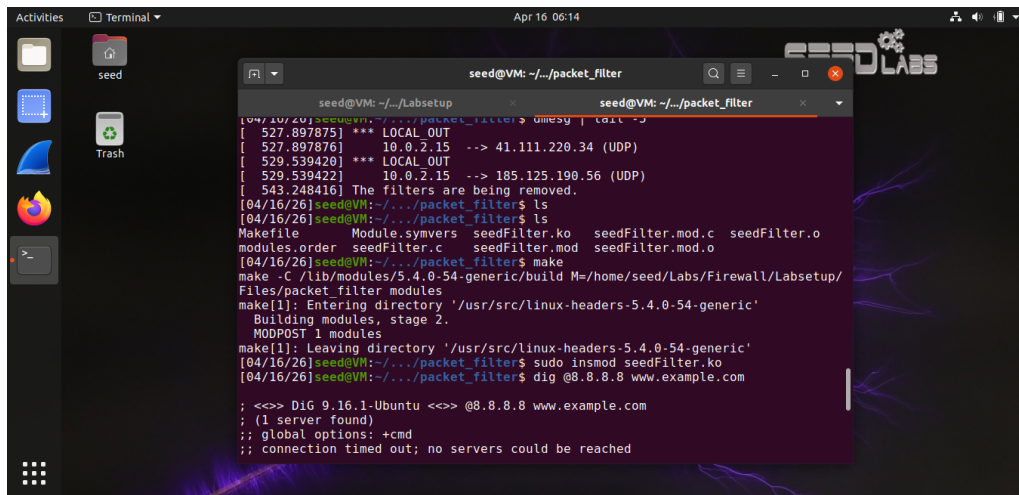


Figure 2: DNS query to 8.8.8.8 times out while `seedFilter` is loaded, confirming the Netfilter hook drops the packets.

2.2.2 Sub-task 2: Hook `printInfo` to All Five Hooks

We register `printInfo` to all five Netfilter hook points and send a ping to observe which hooks fire:

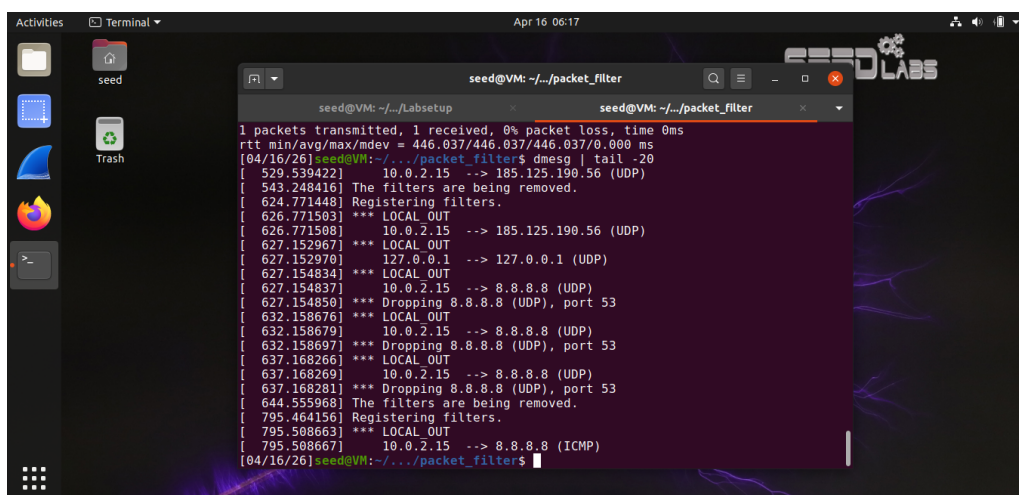
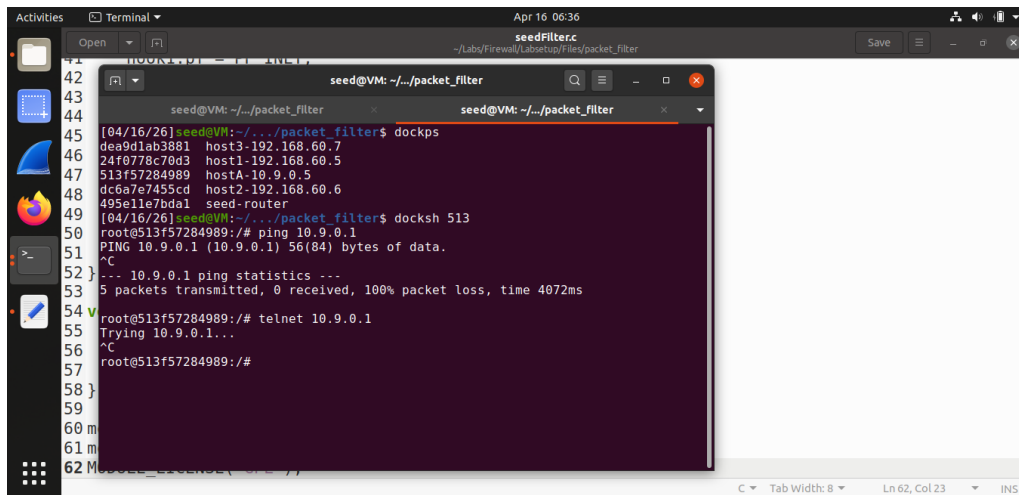


Figure 3: `dmesg` output showing which hooks fire for an outgoing ping: `LOCAL_OUT` and `POST_ROUTING` fire for packets generated locally; `PRE_ROUTING` and `LOCAL_IN` fire for incoming replies; `FORWARD` fires only for packets being routed through.

2.2.3 Sub-task 3: Block Ping and Telnet to VM

Two hook functions are registered to `NF_INET_LOCAL_IN`, which returns the following :



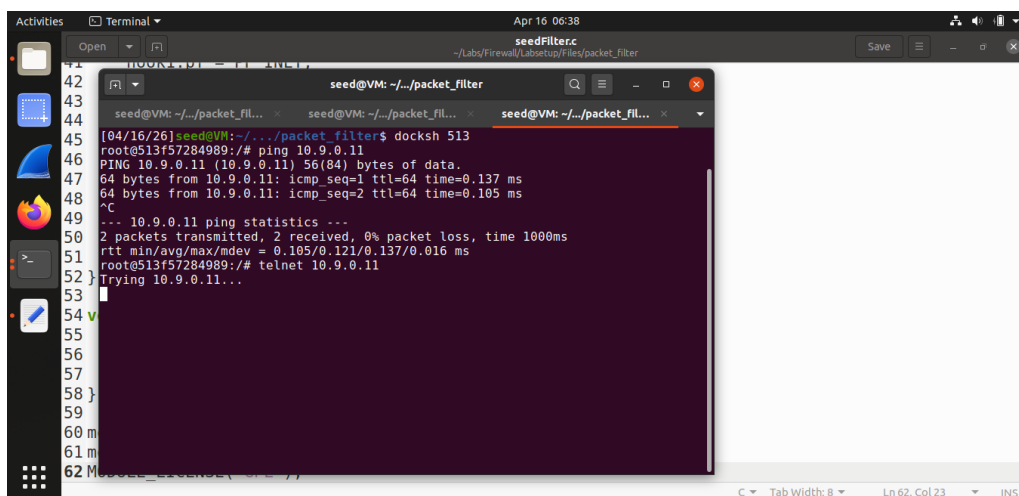
```
seed@VM: ~/.../packet_filter
[04/16/26]seed@VM:~/.../packet_filter$ docksh
de99d1ab3881 host3-192.168.60.7
24f0778c70d3 host1-192.168.60.5
513f57284989 hostA-10.9.0.5
dc6a7e7455cd host2-192.168.60.6
495e11e7bdal seed-router
[04/16/26]seed@VM:~/.../packet_filter$ docksh 513
root@513f57284989:~# ping 10.9.0.1
PING 10.9.0.1 (10.9.0.1) 56(84) bytes of data.
^C
--- 10.9.0.1 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4072ms
root@513f57284989:~# telnet 10.9.0.1
Trying 10.9.0.1...
^C
root@513f57284989:~#
```

Figure 4: Ping from 10.9.0.5 to the VM (10.9.0.1) receives no reply after loading the module. Telnet also fails to connect.

3 Task 2: Stateless Firewall with iptables

3.1 Task 2.A: Protecting the Router

We configure a restrictive firewall policy that blocks all incoming and outgoing network traffic by default. The only allowed traffic is ICMP ping communication, permitting the system to receive ping requests (echo-request) and send ping replies (echo-reply).



```
seed@VM: ~/.../packet_filter
[04/16/26]seed@VM:~/.../packet_filter$ docksh 513
root@513f57284989:~# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.137 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.105 ms
^C
--- 10.9.0.11 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 0.105/0.121/0.137/0.016 ms
root@513f57284989:~# telnet 10.9.0.11
Trying 10.9.0.11...
```

Figure 5: Ping to the router succeeds; telnet is refused, confirming the rules work as intended.

Analysis: Only ICMP echo-request/reply is explicitly allowed. The default DROP policy blocks everything else, including telnet (TCP 23).

3.2 Task 2.B: Protecting the Internal Network

Next we configure packet forwarding between two network interfaces while applying strict filtering rules. Devices on the internal network (eth1) are allowed to send ping requests to external networks and receive ping replies, while ping requests originating from the external network (eth0) toward the internal network are blocked. All other forwarded traffic is denied by the default DROP policy.

```
seed@VM: ~/.../packet_filter
root@513f57284989:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
19 packets transmitted, 0 received, 100% packet loss, time 18415ms
root@513f57284989:/#
```

Figure 6: Ping from 10.9.0.5 to 192.168.60.5 fails (outside cannot ping internal). Ping from 192.168.60.5 to 10.9.0.5 succeeds.

3.3 Task 2.C: Protecting Internal Servers

We allow external hosts to access only the Telnet service (TCP port 23) on the internal server 192.168.60.5, while blocking other external access. Internal network hosts on 192.168.60.0/24 are permitted to communicate freely with internal servers using TCP traffic, and all other forwarded traffic is denied by the default DROP policy.

```
seed@VM: ~/.../packet_filter
root@513f57284989:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
^C
24f0778c70d3 login: ^CConnection closed by foreign host.
root@513f57284989:/# telnet 192.168.60.6
Trying 192.168.60.6...
^C
root@513f57284989:/#
```

Figure 7: Telnet from 10.9.0.5 to 192.168.60.5 succeeds; telnet to 192.168.60.6 is blocked. Internal hosts can reach each other freely.

4 Task 3: Stateful Firewall

4.1 Task 3.A: Connection Tracking

We run `conntrack -L` on the router after each protocol test.

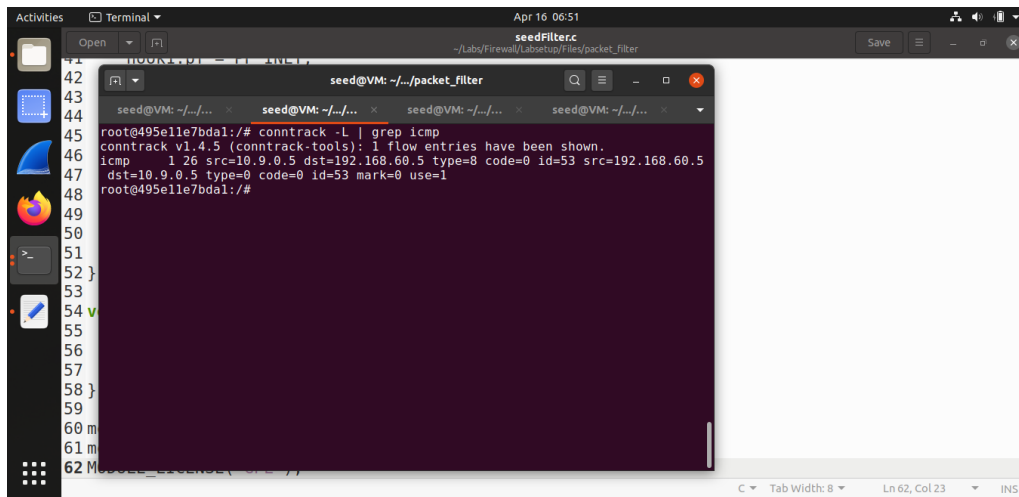
A terminal window titled 'seed@VM: ~/packet_filter' shows the output of the command 'conntrack -L | grep icmp'. The output displays a single flow entry for ICMP: 'icmp 1 26 src=10.9.0.5 dst=192.168.60.5 type=8 code=0 id=53 src=192.168.60.5 dst=10.9.0.5 type=0 code=0 id=53 mark=0 use=1'. The terminal is part of a larger window titled 'seedFilter.c' with a file explorer on the left showing a directory structure with files like '42', '43', '44', '45', '46', '47', '48', '49', '50', '51', '52', '53', '54', '55', '56', '57', '58', '59', '60', '61', '62'.

Figure 8: ICMP connection entry visible in `conntrack` output during ping. State expires after approximately 30 seconds of inactivity.

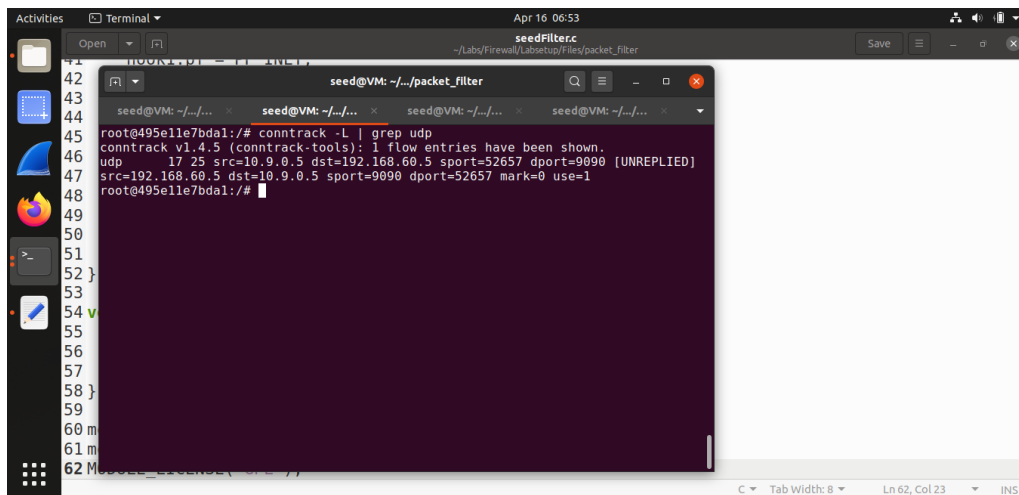
A terminal window titled 'seed@VM: ~/packet_filter' shows the output of the command 'conntrack -L | grep udp'. The output displays a single flow entry for UDP: 'udp 17 25 src=10.9.0.5 dst=192.168.60.5 sport=52657 dport=9090 [UNREPLIED] src=192.168.60.5 dst=10.9.0.5 sport=9090 dport=52657 mark=0 use=1'. The terminal is part of a larger window titled 'seedFilter.c' with a file explorer on the left showing a directory structure with files like '42', '43', '44', '45', '46', '47', '48', '49', '50', '51', '52', '53', '54', '55', '56', '57', '58', '59', '60', '61', '62'.

Figure 9: UDP connection tracked despite being connectionless. Entry shows UNREPLIED or ASSURED state and expires after 30 seconds.

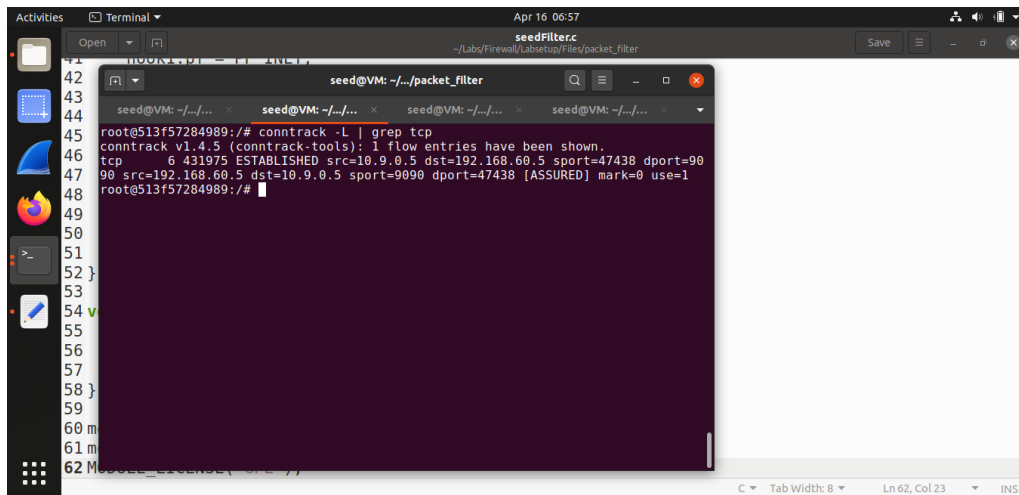


Figure 10: TCP connection shows **ESTABLISHED** state while active. Entry persists until the connection is closed, after which it transitions to **TIME_WAIT**.

4.2 Task 3.B: Stateful Firewall Rules

Then we implement a stateful firewall using connection tracking (`conntrack`). Internal hosts connected through `eth1` are allowed to initiate new outbound TCP connections, while external hosts may only start new Telnet connections (TCP port 23) to the server 192.168.60.5. Established and related traffic is permitted so active connections can continue normally, and all other forwarded traffic is blocked by the default DROP policy.

Advantage over Task 2.C: The **ESTABLISHED,RELATED** rule replaces the need for explicit reverse-direction rules. Internal hosts can now reach external servers freely (any new connection from `eth1` is allowed). Without `conntrack`, Task 2.C required manually permitting both directions of every allowed flow, which is error-prone and cannot handle dynamic port assignments.

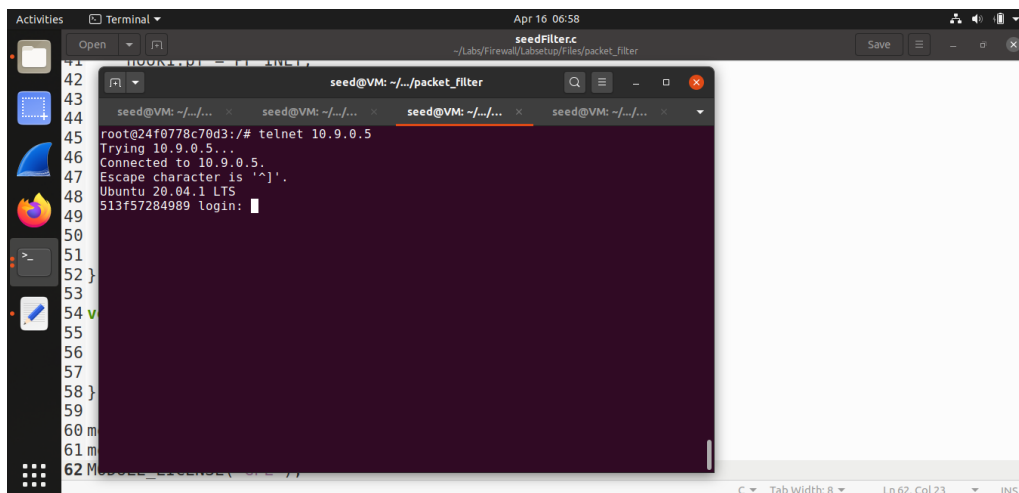


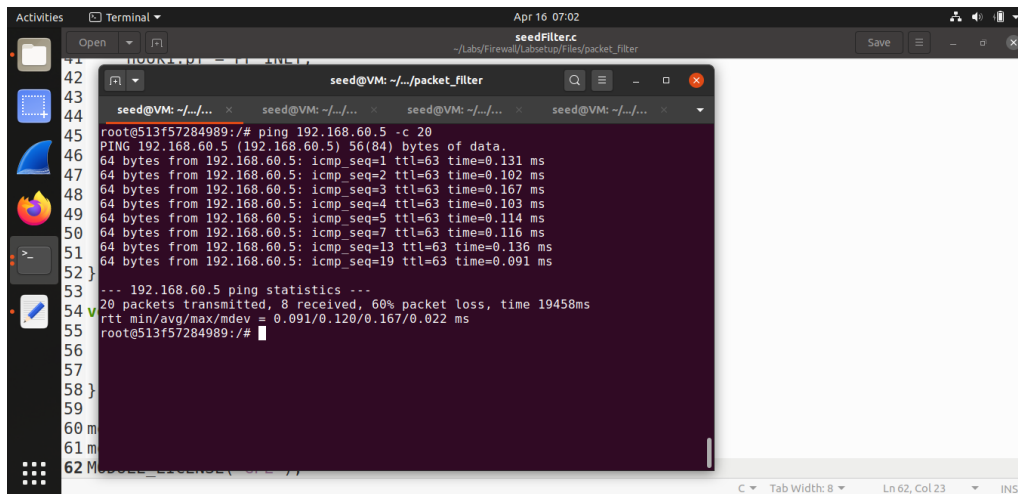
Figure 11: Stateful firewall correctly allows established connections and internal-initiated outbound traffic while blocking unsolicited inbound connections.

5 Task 4: Limiting Network Traffic

```
iptables -A FORWARD -s 10.9.0.5 \
-m limit --limit 10/minute --limit-burst 5 -j ACCEPT
```

```
iptables -A FORWARD -s 10.9.0.5 -j DROP
```

The first rule allows a burst of 5 packets immediately, then limits to 10 packets per minute. The second rule drops everything from 10.9.0.5 that exceeds the limit. Without the second rule, excess packets would fall through to the default ACCEPT policy and be forwarded normally, making the limit ineffective.



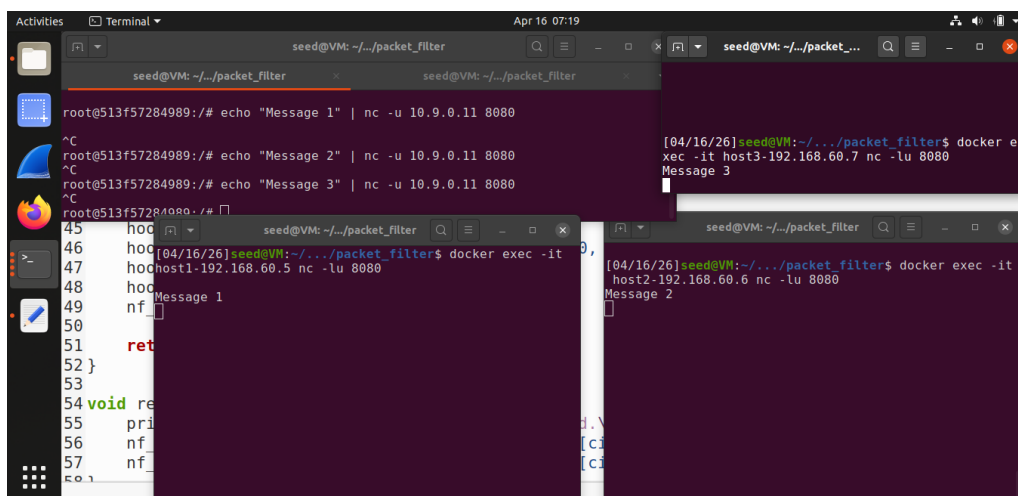
```
seed@VM: ~/.../packet_filter
root@513f57284989:/# ping 192.168.60.5 -c 20
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data:
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.131 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.102 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.167 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.103 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=0.114 ms
64 bytes from 192.168.60.5: icmp_seq=7 ttl=63 time=0.116 ms
64 bytes from 192.168.60.5: icmp_seq=13 ttl=63 time=0.136 ms
64 bytes from 192.168.60.5: icmp_seq=19 ttl=63 time=0.091 ms
--- 192.168.60.5 ping statistics ---
20 packets transmitted, 8 received, 60% packet loss, time 19458ms
rtt min/avg/max/mdev = 0.091/0.120/0.167/0.022 ms
root@513f57284989:/#
```

Figure 12: Ping from 10.9.0.5 shows first 5 replies arrive quickly (burst), then packets are dropped until the rate drops below 10/minute.

6 Task 5: Load Balancing

6.1 nth Mode (Round-Robin)

The used configuration uses iptables NAT rules to perform basic load balancing for UDP traffic on port 8080. The statistic module distributes incoming packets across three internal servers by matching packets in a repeating pattern, ensuring that each server (192.168.60.5, 192.168.60.6, and 192.168.60.7) receives approximately one-third of the total traffic.



```
seed@VM: ~/.../packet_filter
root@513f57284989:/# echo "Message 1" | nc -u 10.9.0.11 8080
^C
root@513f57284989:/# echo "Message 2" | nc -u 10.9.0.11 8080
^C
root@513f57284989:/# echo "Message 3" | nc -u 10.9.0.11 8080
^C
root@513f57284989:/#
```

```
seed@VM: ~/.../packet_filter
[04/16/26]seed@VM:~/.../packet_filter$ docker exec -it host3-192.168.60.7 nc -lu 8080
Message 3
```

```
seed@VM: ~/.../packet_filter
[04/16/26]seed@VM:~/.../packet_filter$ docker exec -it host1-192.168.60.5 nc -lu 8080
Message 1
```

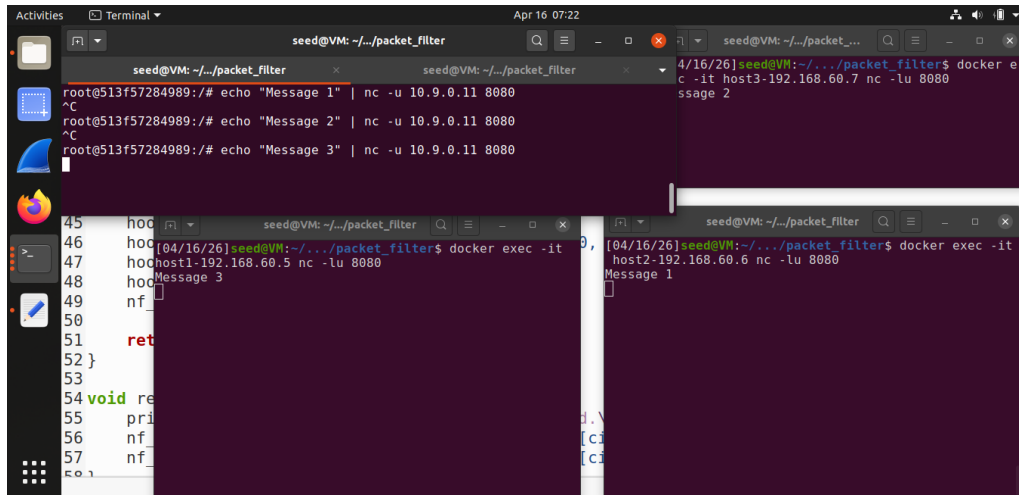
```
seed@VM: ~/.../packet_filter
[04/16/26]seed@VM:~/.../packet_filter$ docker exec -it host2-192.168.60.6 nc -lu 8080
Message 2
```

Figure 13: 9 UDP packets sent; each of the three servers receives exactly 3 packets, confirming perfect round-robin distribution.

6.2 Random Mode

Another configuration that uses iptables NAT rules with the statistic module in random mode to distribute UDP traffic arriving on port 8080 across three internal servers. Approximately 33% of packets are redirected to 192.168.60.5, around half of the remaining traffic is sent to 192.168.60.6, and all remaining packets are forwarded to 192.168.60.7, resulting in an overall near-even load distribution.

Logic: Rule 1 catches 33% of all packets. Of the remaining 67%, rule 2 catches 50% ($\approx 33\%$). The rest ($\approx 33\%$) goes to host 7. Distribution is approximate and converges with large packet counts.



```
seed@VM: ~/.../packet_filter
root@513f57284989:/# echo "Message 1" | nc -u 10.9.0.11 8080
^C
root@513f57284989:/# echo "Message 2" | nc -u 10.9.0.11 8080
^C
root@513f57284989:/# echo "Message 3" | nc -u 10.9.0.11 8080
^C

45 hoo
46 hoo
47 hoo[04/16/26]seed@VM:~/.../packet_filter$ docker exec -it host1-192.168.60.5 nc -lu 8080
48 hooMessage 3
49 nf
50 nf
51 ret
52 }
53
54 void re
55 pri
56 nf
57 nf
58 so

[04/16/26]seed@VM:~/.../packet_filter$ docker exec -it host2-192.168.60.6 nc -lu 8080
Message 1
[04/16/26]seed@VM:~/.../packet_filter$ docker exec -it host3-192.168.60.7 nc -lu 8080
ssage 2
```

Figure 14: Random mode distribution over many packets. Each server receives approximately equal traffic, with minor variance due to probabilistic selection.

7 Conclusion

This lab demonstrated two complementary firewall approaches. LKM-based Netfilter hooks provide low-level packet inspection at kernel speed but require C programming and careful module management to avoid kernel crashes. `iptables` provides a higher-level, manageable interface covering stateless filtering, stateful connection tracking, rate limiting, and NAT-based load balancing. Stateful firewalls using `conntrack` are significantly simpler to configure correctly than equivalent stateless rules, at the cost of maintaining per-connection state in the kernel.